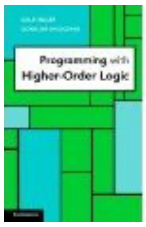
Programming with Higher-Order Logic

by Dale Miller and Gopalan Nadathur.
Published by Cambridge University Press, June 2012.



Programming with Higher-order Logic focuses on how logic programming can exploit higher-order intuitionistic logic. The authors emphasize using higher-order logic programming to declaratively specify a range of applications.

All code fragments from this book
Below is a version of the book in which most of the code and sectioning information (chapters, sections, subsection) is kept. You can use this long file to locate almost all the program and goals fragments that appear in the book. You can navigate this file by using the table-of-contents below and by using the search feature of your browser to locate keywords. You can then use cut-and-paste to move code from this web page into your own files of λProlog code. The code here has been written for and tested with the Teyjus compiler.

Contents

Chapter 1 First-Order Terms and Representations of Data

1 Sorts and type constructors

```
kind   pair   type -> type -> type.
```

2 Type expressions

3 Typed first-order terms

```
type    pr     A -> B -> pair A B.

infixl   pr   5.
```

4 Representing symbolic objects

4.1 Representing binary trees

```
kind   btree   type -> type.

type   empty   btree A.
type   node    A -> btree A -> btree A -> btree A.

kind   btree   type.
type   empty   btree.
type   node    A -> btree -> btree -> btree.
```

4.2 Representing logical formulas

```
kind   term, form   type.

type   a, b   term.
type   f      term -> term -> term.

type   var   string -> term.

type   p   term -> term -> form.
type   q   term -> form.

type    ff,                          % encoding the false proposition
        tt   form.                   % encoding the true proposition
type    &&,                          % encoding conjunction
        !!,                          % encoding disjunction
        ==>  form -> form -> form.   % encoding implication
infixl  &&  5.
infixl  !!  4.
infixr  ==> 3.
```

```
type   all   term -> form -> form.
```

4.3 Representing imperative programs

```
kind stmt, expr     type.
type id             string -> expr.
type c              int -> expr.
type t, f           expr.
type &&, !!,
     plus, minus,
     mult, <        expr -> expr -> expr.
type :=             expr -> expr -> stmt.            % for assignment
type cond           expr -> stmt -> stmt -> stmt.    % for conditionals
type while          expr -> stmt -> stmt.            % for while loops
type seq            stmt -> stmt -> stmt.            % for composition
infixl &&, mult     5.
infixl !!,
       plus, minus  4.
infix  <            3.
infix :=            2.
```

```
   v = 1; i = n;
   while (0 < i) {
     v = v * i; i = i - 1;
   }
```

```
  (seq ((id "v") := (c 1))
       (seq ((id "i") := (id "n"))
            (while ((c 0) < (id "i"))
                   (seq ((id "v") := (id "v") mult (id "i"))
                        ((id "i") := (id "i") minus (c 1))))))
```

5 Unification of typed first-order terms

6 Bibliographic notes

Chapter 2 First-Order Horn Clauses

1 First-order formulas

```
type  memb       A -> list A -> o.
type  append     list A -> list A -> list A -> o.
```

```
memb 1 (1 :: 2 :: nil)
append (1 :: nil) (2 :: nil) (1 :: 2 :: nil)
```

```
(memb 1 (2 :: 1 :: nil) :- memb 1 (1 :: nil)), memb 1 (1 :: nil)
memb 1 (1 :: nil) => memb 1 (2 :: 1 :: nil) & memb 1 (1 :: nil)
(memb 1 (1 :: nil) => memb 1 (2 :: 1 :: nil)) & memb 1 (1 :: nil)
(append nil nil nil ; memb 1 (2 :: nil)), append (1 :: nil) nil (1 :: nil)
```

```
(pi x\ (pi z\ (append x z x =>
              (sigma y\ (append x y z, pi x\ (append y z x))))))).


pi x\ pi z\ append x z x => sigma y\ append x y z, pi x\ append y z x.


pi x\ pi y\ pi z\ append x y z


pi x\ pi k\ memb x (x :: k)
pi X\ pi L\ pi K\ pi M\ append (X::L) K (X::M) :- append L K M
sigma X\ pi y\ sigma h\ append X y h


pi x\ (p x) => sigma y\ (q x y, pi x\ (q y x))
pi x\ (p x) => sigma U\ (q x U, pi x\ (q U x))
pi z\ (p z) => sigma y\ (q z y, pi x\ (q y x))
pi z\ (p z) => sigma y\ (q z y, pi v\ (q y v))
```

2 Logic programming and search semantics

3 Horn clauses and their computational interpretation

4 Programming with first-order Horn clauses

4.1 Concrete syntax for program clauses

```
type  append  list A -> list A -> list A -> o.


append nil L L.
append (X :: L1) L2 (X :: L3) :- append L1 L2 L3.


pi L\ append nil L L.
pi X\ pi L1\ pi L2\ pi L3\
             append (X :: L1) L2 (X :: L3) :- append L1 L2 L3.


pi l\ append nil l l.
pi x\ pi l1\ pi l2\ pi l3\
          append (x :: l1) l2 (x :: l3) :- append l1 l2 l3.


type sublist   list A -> list A -> o.
sublist L K :- append _ T K, append L _ T.


sublist L K :- append U T K, append L V T.


kind  bool          type.
type  neg           bool -> bool.
type  and, or, imp  bool -> bool -> bool.
type  ident         bool -> bool -> o.
```

```
ident (neg B)   (neg D)   :- ident B D.
ident (and B C) (and D E) :- ident B D, ident C E.
ident (or  B C) (or  D E) :- ident B D, ident C E.
ident (imp B C) (imp D E) :- ident B D, ident C E.


ident (neg B)   (neg D)   :- ident B D.
ident (and B C) (and D E) &
ident (or  B C) (or  D E) &
ident (imp B C) (imp D E) :- ident B D, ident C E.


ident (neg B)    (neg D)   &
(ident (and B C) (and D E) &
 ident (or  B C) (or  D E) &
 ident (imp B C) (imp D E) :- ident C E) :- ident B D.
```

4.2 Interacting with the λProlog system

```
module lists.
type  append    list A -> list A -> list A -> o.
append nil L L.
append (X::L) K (X::M) :- append L K M.
end


[lists] ?-


[lists] ?- sigma X\ sigma Y\ append X Y (1 :: 2 :: nil).


[lists] ?- sigma Y\ append X Y (1::nil).
[lists] ?- append X Y (1::nil).


[lists] ?- append _ _ (1::nil).


[lists] ?- append (1::nil) (2::nil) (3::nil).
no

[lists] ?-


[lists] ?- append (1::nil) (2::nil) (1::2::nil).
solved

[lists] ?-


[lists] ?- append (1::nil) (2::nil) X.
X = (1::2::nil)

[lists] ?-
```

```
[lists] ?- sigma X\ append (1::nil) (2::nil) X.
solved

[lists] ?- append (1::nil) (2::nil) _.
solved

[lists] ?-


[lists] ?- sigma Y\ append X Y (1::nil).
X = nil

[lists] ?- append X _ (1::nil).
X = nil

[lists] ?- append X Y (1::nil).
X = nil
Y = (1::nil)

[lists] ?-


[lists] ?- append X Y (1::nil).
X = nil
Y = 1::nil;

X = 1::nil
Y = nil;

no

[lists] ?-
```

4.3 Reachability in a finite state machine

```
kind state, letter         type.
type q1, q2, q3, q4, q5    state.
type a, b                  letter.
type start, final          state -> o.
type path, trans           state -> list letter -> state -> o.
type accept                list letter -> o.

path S nil S.
path S Letters T :- trans S Arc M, append Arc Rest Letters,
                    path M Rest T.
accept W :- start S, path S W F, final F.


start q1 & final q2 & final q3.
trans q1 (a::nil) q1    & trans q1 (b::nil) q1.
trans q1 (a::b::nil) q2 & trans q1 (b::a::nil) q3.


start q1 & final q4 & final q5.
```

```
trans q1 (a::nil) q2   &   trans q1 (b::nil) q3.
trans q2 (a::nil) q2   &   trans q2 (b::nil) q4.   % Correction made here
trans q3 (a::nil) q5   &   trans q3 (b::nil) q3.   % Correction made here
trans q4 (a::nil) q5   &   trans q4 (b::nil) q3.
trans q5 (a::nil) q2   &   trans q5 (b::nil) q4.


[fsm1] ?- accept (b::b::a::b::nil).
solved

[fsm1] ?- accept (b::a::X::Y::nil).
X = a
Y = b ;

X = b
Y = a ;

no

[fsm1] ?-


type lists   list A -> o.
lists nil.
lists (_::L) :- lists L.


[fsm1] ?- lists L.
L = nil ;

L = T1 :: nil ;

L = T1 :: T2 :: nil ;

L = T1 :: T2 :: T3 :: nil

[fsm1] ?- lists L, accept L.
L = a :: b :: nil ;

L = b :: a :: nil ;

L = a :: a :: b :: nil ;

L = a :: b :: a :: nil ;

L = b :: a :: b :: nil

[fsm1] ?-
```

4.4 Defining relations over recursively structured data

```
append nil L L.
```

```
append (X :: L1) L2 (X :: L3) :- append L1 L2 L3.


kind  btree    type -> type.
type  empty    btree A.
type  node     A -> btree A -> btree A -> btree A.


type    insert    int -> btree int -> btree int -> o.
insert X empty (node X empty empty).
insert X (node A L R) (node A NL R) :- X < A, insert X L NL.
insert X (node A L R) (node A L NR) :- X >= A, insert X R NR.


[btree] ?- insert 4 (node 3 (node 2 empty empty) empty) T.
T = node 3 (node 2 empty empty) (node 4 empty empty)

[btree] ?-
```

4.5 Programming over abstract syntax representations

```
kind  term, form   type.
type  ff, tt       form.
type  &&, !!, ==>  form -> form -> form.
infixl &&          5.
infixl !!          4.
infixr ==>         3.
type  a,b          term.
type  f            term -> term -> term.
type  p,q          term -> term -> form.


type  memb_and_rest   A -> list A -> list A -> o.
memb_and_rest X (X :: L) L.
memb_and_rest X (Y :: L) (Y :: L1) :- memb_and_rest X L L1.


type  prv    list form -> list form -> o.


prv Gamma Delta :- memb_and_rest ff Gamma _.
prv Gamma Delta :- memb_and_rest A Gamma _,
                   memb_and_rest A Delta _.
prv Gamma Delta :-
      memb_and_rest (A && B) Gamma Gamma',
      prv (A :: B :: Gamma') Delta.
 prv Gamma Delta :-
      memb_and_rest (A !! B) Gamma Gamma',
      prv (A :: Gamma') Delta, prv (B :: Gamma') Delta.
prv Gamma Delta :-
      memb_and_rest (A ==> B) Gamma Gamma',
      prv Gamma' (A :: Delta), prv (B :: Gamma') Delta.
prv Gamma Delta :-
      memb_and_rest (A && B) Delta Delta',
```

```
      prv Gamma (A :: Delta'), prv Gamma (B :: Delta').
prv Gamma Delta :-
      memb_and_rest (A !! B) Delta Delta',
      prv Gamma (A :: B :: Delta').
prv Gamma Delta :-
      memb_and_rest (A ==> B) Delta Delta',
      prv (A :: Delta) (B :: Gamma').


[logic] ?- prv nil (((p a b) !! ((p a b) ==> (q a a))) :: nil).
solved

[logic] ?-
```

5 Pragmatic aspects of computing with Horn clauses

6 The relationship to logical notions

7 The meaning and use of types

7.1 Types and the categorization of expressions

7.2 Polymorphic typing

```
kind lst    type.
type null   lst.
type cons    A -> lst -> lst.
```

7.3 Type checking and type inference

```
append (1::nil) (2::nil) X, append ("abc"::nil) ("efg"::nil) Y.


list int -> list int -> list int -> o
list string -> list string -> list string -> o.


pi (X:list int)\ append X X Y.


append (X:list int) X Y.
```

7.4 Types and run-time computations

```
kind lst          type.
type null         lst.
type cons         A -> lst -> lst.
type separate     lst -> list int -> list real -> o.

separate (cons (X:int)  L) (X::K) M :- separate L K M.
separate (cons (X:real) L) K (X::M) :- separate L K M.
separate null nil nil.


kind numb         type.
type inj_int      int  -> numb.
```

```
type inj_real     real -> numb.
type separate     list numb -> list int -> list real -> o.

separate ((inj_int  X)::L) (X::K) M :- separate L K M.
separate ((inj_real X)::L) K (X::M) :- separate L K M.
separate nil nil nil.


separate (cons 1.0 (cons 2 (cons 3.0 null))) L K


separate ((inj_real 1.0)::(inj_int 2)::(inj_real 3.0)::nil) L K
```



```
?- p 2 => p 3 => p X.
X = 3;
X = 2;
X = 1

?- (p 2 & p 3) => p X.
X = 2;
X = 3;
X = 1

?-
```



```
s :- r, q.
t :- q, u.
q :- r.


t :- r, u.


s :- r.


?- s :- r.


?- r => s.


(r => u) => (r => t)
```

```
?- (q :- (q => q)) => (q :- (q => q)).
```

2.2 Hypothetical reasoning

```
?- fact (finished dana X) => false.
X = 210;
no

?-


?-  fact (finished X Y) => (fact (graduates X), fact (cs_major X)).
X = dana
Y = 301;
no

?-


?-  fact (finished kim X) => fact (finished kim Y) =>
                        (fact (graduates kim), fact (cs_major kim)).
X = 250
Y = 301

?-


?-  fact (finished kim 250) => fact (finished kim 301) => false.
solved

?-


kind entry                  type.
type fact                   entry -> o.
type false                  o.
kind person                 type.
type kim, dana              person.
type finished               person -> int -> entry.
type cs_major, graduates    person -> entry.

fact (finished kim  102) & fact (finished dana 101).
fact (finished kim  210) & fact (finished dana 250).

fact (cs_major X)  :-
     (fact (finished X 101); fact (finished X 102)),
     fact (finished X 250), fact (finished X 301).
fact (graduates X) :-
     (fact (finished X 101); fact (finished X 102)),
     (fact (finished X 210); fact (finished X 250)),
     fact (finished X 301).

false :- fact (finished X 210), fact (finished X 250).
```

```
type db                              o.
kind command                         type.
type do                              command -> o.
type enter, query, whatif, check     entry -> command.
type quit, consis                    command.

db :- print "Command?", read Command, do Command.

do quit.
do (enter Fact) :- fact Fact => db.
do (query Q) :- (fact Q, !, print "yes\n"; print "no\n"), db.
do (whatif Conjecture) :- (fact Conjecture => db),
                          print "Resuming\n", db.
do consis :- false, print "no\n", !; print "yes\n".
do (check Entry) :- (fact Entry, print "yes\n", !;
                     fact Entry => false, print "no\n", !;
                     print "no, but it could be true\n"),  db.


?- db.
Command? query (graduates dana).
no
Command? whatif (finished dana 301).
Command? query (graduates dana).
yes
Command? query (cs_major dana).
yes
Command? quit.
Resuming.
Command? query (finished kim 101).
no
Command? query (graduates kim).
no
Command? whatif (finished kim 301).
Command? query (graduates kim).
yes
Command? query (cs_major kim).
no
Command? whatif (finished kim 250).
Command? query (cs_major kim).
yes
Command? consis.
no
Command?
```

3 Universally quantified goals

```
kind jar, bug           type.
type j                  jar.
type sterile, heated    jar -> o.
type dead, bug          bug -> o.
type in                 bug -> jar -> o.
```

```
sterile J :- pi x\ bug x => in x J => dead x.
dead B    :- heated J, in B J, bug B.
heated j.


pi x\ bug x => in x j => dead x.


bug g => in g j => dead g.


kind nat     type.
type zero    nat.
type succ    nat -> nat.
type plus    nat -> nat -> nat -> o.

plus zero L L.
plus (succ N) M (succ P) :- plus N M P.


?- pi N\ plus N zero N.


?- plus zero zero zero,
   pi N\ plus N zero N =>  plus (succ N) zero (succ N).
```

3.1 Substitution and quantification

```
p X :- pi y\ q X y.


p (f y) :- pi y\ q (f y) y.


p (f y) :- pi z\ q (f y) z.


?- sigma x\ pi y\ x = y.
```

3.2 Quantification can link goals and clauses

```
?- reverse (1::2::nil) P.


?- rev (1::2::nil) P nil.


rev nil L L.
rev (X::L) K M :- rev L K (X::M).


type reverse    list A -> list A -> o.
type rev        list A -> list A -> list A -> o.

reverse L K :-
```

```
   ((pi L\ rev nil L L) &
    (pi X\ pi L\ pi K\ pi M\ rev (X::L) K M :- rev L K (X::M)))
          => rev L K nil.


type reverse, rev    list A -> list A -> o.

reverse L K :-
   (rev nil K &
    (pi X\ pi L\ pi K\ rev (X::L) K :- rev L (X::K)))
          => rev L nil.


rv nil (c::b::a::nil).
rv (X::N) M :- rv N (X::M).


?- reverse (1::2::nil) P.


?- rev (1::2::nil) nil.


rev nil K.
rev (X::L) K :- rev L (X::K).
```

4 The relationship to logical notions

```
loop :- loop.
```

4.1 Classical vs intuitionistic logic
4.2 Intuitionistic vs minimal logic

```
(p => q) => ((q => false) => (p => false)).
p => ((p => false) => false).


?- p; (p => false).


?- ((p; (p => false)) => false) => false.


   (p; (p => false)) => false ?- false.
   (p; (p => false)) => false ?- p; (p => false).
   (p; (p => false)) => false ?- p => false.
p, (p; (p => false)) => false ?- false.
p, (p; (p => false)) => false ?- p; (p => false).
p, (p; (p => false)) => false ?- p.
```

4.3 Notable subsets of fohh

5 Bibliographic notes

Chapter 4 Typed λ-Terms and Formulas

1 Syntax for λ-terms and formulas

```
y\ append (1::2::nil) y X.

pi y\ append (1::2::nil) y X.

type foreach   (A -> o) -> list A -> o.

foreach (x\ x > 5, x < 9) (3::10::6::8::nil).
```

2 The rules for λ-conversion

```
x\y\ f (g x) y
X\Y\ f (g X) Y
x\ f (g x)
x\y\ f ((u\v\v) (2 + 3) (g x)) y

x\y\ f ((v\v) (g x)) y
```

3 Some properties of λ-conversion

```
?- N = ((n\m\f\x\ n (m f) x) ((f:i -> i)\x\ f (f x)) (f\x\ f (f x))).
N = f\x\ f (f (f (f x)))

?-

(g\e\ e)           (e\f\ e (e f)) (f\x\ f (f x)).
(g\e\ g e)         (e\f\ e (e f)) (f\x\ f (f x)).
(g\e\ g (g e))     (e\f\ e (e f)) (f\x\ f (f x)).
(g\e\ g (g (g e))) (e\f\ e (e f)) (f\x\ f (f x)).

x : i
f : i -> i
e : (i -> i) -> i -> i
g : ((i -> i) -> i -> i) -> (i -> i) -> i -> i.

(((i -> i) -> i -> i) -> (i -> i) -> i -> i) ->
 ((i -> i) -> i -> i) -> (i -> i) -> i -> i.

    f\x\ f (f (f (f (f (f (f (f (f (f (f
                          (f (f (f (f x))))))))))))))
```

4 Unification problems as quantified equalities

```
?- pi y\ X = y.
no

?-


?-  (y\ X) = (y\ y).
no

?-
```


Chapter 5 Using Quantification at Higher-Order Types


```
P 5 :- q.
q.


P (X + 0) :- P X.
P X :- P (X + 0).


?- convert d P, P => g.
```



```
type reverse   list A -> list A -> o.

reverse L K :-  pi rev\
  (pi L\ rev nil L L) &
  (pi X\ pi L\ pi K\ pi M\ rev (X::L) K M :- rev L K (X::M))
   => rev L K nil.


type reverse   list A -> list A -> o.

reverse L K :- pi rv\
  (                   rv nil K &
   (pi X\ pi N\ pi M\ rv (X::N) M :- rv N (X::M)))
   => rv L nil.
```

```
pi L\ c nil L L.
pi X\ pi L\ pi K\ pi M\ c (X::L) K M :- c L K (X::M).
```

3 Examples of higher-order programming

```
type foreach, forsome   (A -> o) -> list A -> o.
type mappred            (A -> B -> o) -> list A -> list B -> o.
type sublist            (A -> o) -> list A -> list A -> o.

foreach P nil.
foreach P (X::L) :- P X, foreach P L.

forsome P (X::L) :- P X; forsome P L.

mappred P nil nil.
mappred P (X::L) (Y::K) :- P X Y, mappred P L K.

sublist P (X::L) (X::K) :- P X, sublist P L K.
sublist P (X::L) K :- sublist P L K.
sublist P nil nil.


kind name             type.
type bob, sue, ned    name.
type age              name -> int -> o.
type male, female     name -> o.

age  bob 23 & age sue 24 & age  ned 23.
male bob    & female sue & male ned.


?- mappred age (ned::bob::sue::nil) L.
L = (23::23::24::nil)

?- mappred age L (23::24::nil).
L = (bob::sue::nil);

L = (ned::sue::nil)

?- sublist male (ned::bob::sue::nil) L.
L = ned::bob::nil;

L = ned::nil;

L = bob::nil;

L = nil;
no

?- forsome female (ned::bob::sue::nil).
solved

?- foreach female (ned::bob::sue::nil).
```

```
no

?-


type ref, sym, trans  (A -> A -> o) -> A -> A -> o.

ref   R X Y :- X = Y; R X Y.
sym   R X Y :- R X Y; R Y X.
trans R X Y :- R X Y.
trans R X Z :- R X Y, trans R Y Z.


kind node  type.
type a, b, c, d, e    node.
type adj              node -> node -> o.

adj a b & adj b c & adj b d & adj d c & adj c e.


?- trans adj a d.
solved

?- sym adj b a.
solved

?-


x\y\ pi p\
  (pi U\ pi V\ adj U V => p U V) =>
  (pi U\ pi V\ pi W\ adj U V => p V W => p U W) =>  p x y


     x\ age jane x            n\ age n 24


?- (n\ age n 24) W.


?- age W 24.


?- mappred (x\y\ age x y) (ned::bob::sue::nil) L.
L = (23::23::24::nil)

?- mappred (x\y\ age y x) (23::24::nil) K.
K = (bob::sue::nil);

K = (ned::sue::nil)

?- foreach (x\ sigma y\ age x y) (ned::bob::sue::nil).
solved

?- foreach (x\ age x A) (ned::bob::sue::nil).
no
```

```
?-


?- foreach (x\ age x A) (ned::bob::nil).
A = 23

?-


x\y\ R x y; S x y.


x\z\ sigma Y\ R x Y, S Y z.


A -> B -> B -> o,


list A -> B -> B -> o.


type union          (A -> B -> o) -> (A -> B -> o) -> A -> B -> o.
type compose        (A -> B -> o) -> (B -> C -> o) -> A -> C -> o.
type foldl          (A -> B -> B -> o) -> list A -> B -> B -> o.

union   R S X Y :- R X Y; S X Y.
compose R S X Y :- R X Z, S Z Y.

foldl P nil    X X.
foldl P (Z::L) X Y :- P Z X W, foldl P L W Y.


kind stack           type -> type.
type emp             stack A.
type stk             A -> stack A -> stack A.
type empty           stack A -> o.
type enter, remove   A -> stack A -> stack A -> o.
type reverse         list A -> list A -> o.

empty  emp.
enter  X S (stk X S).
remove X (stk X S) S.

reverse L K :- compose (foldl enter L) (foldl remove K) emp emp.


adj a b.
adj b c.
path X Y :- adj X Y.
path X Y :- adj X Z, path Z Y.


adj' a b K :- K.
adj' b c K :- K.
path' X Y K :- adj' X Y K.
path' X Y K :- adj' X Z (path' Z Y K).
```

```
?- path X Y.


?- path' X Y true.


?- fib_memo 20 (fib\ sigma M\ fib N M, M is N * N).


type fib_memo   int -> ((int -> int -> o) -> o) -> o.

fib_memo N G :-
   pi memo\ pi loop\
       memo 0 0 => memo 1 1 =>
       (pi F1\ pi F2\ pi F3\ pi C\ pi C'\
          (loop C F1 F2 :- (C >  N, (G memo)) ;
                           (C =< N, C' is C + 1, F3 is F1 + F2,
                            memo C F3 => loop C' F2 F3))) =>
      loop 2 0 1.
```

4 Flexible atoms as goals

```
x\y\ age x 23, age ned y


?- rel R, R john mary.


x\y\ sigma Z\ wife x Z, mother Z y.


kind i                                  type.
type jane, mary, john                   i.
type mother, father, wife, husband      i -> i -> o.
type primrel, rel                       (i -> i -> o) -> o.

primrel father & primrel mother & primrel wife & primrel husband.

rel R :- primrel R.
rel (x\y\ sigma z\ R x z, S z y) :- primrel R, primrel S.

mother jane mary & wife john jane.
```

5 Reasoning about higher-order programs

```
   (a :: b :: c :: nil)   nil
   (b :: c :: nil)  (a :: nil)
   (c :: nil)  (b :: a :: nil)
    nil   (c :: b :: a :: nil)


 pi rv\ ( rv nil K &
```

```
          (pi X\ pi N\ pi M\ rv (X::N) M :- rv N (X::M)))
        => rv L nil


  not(rv K nil) &
          (pi X\ pi N\ pi M\ not(rv M (X::N)) :- not(rv (X::M) N))
        => not(rv nil L).


   rv nil L & (pi X\ pi N\ pi M\ rv (X::M) N :-  rv M (X::N))
           => rv K nil.
```

6 Defining some of the logical constants

```
type tt, ff  o.
type or      o -> o -> o.
type exists  (A -> o) -> o.

tt.                % true
or P Q :- P.       % disjunction
or P Q :- Q.
exists B :- B T.   % existential quantifier
```

7 The conditional and negation-as-failure

```
type if    o -> o -> o -> o.
if P Q R :- P, !, Q.
if P Q R :- R.


type not   o -> o.
not P :- P, !, fail.
not P.


not P :- if P fail true.


?- X = 2, not (1 = X).


?- not (1 = X), X = 2.
```

8 Using λ-terms as functions

8.1 Some basic computations with functional expressions

```
?- mapfun (x\ g a x) (a::b::nil) L.


L = ((g a a)::(g a b)::nil).


mapfun F L K :- mappred (x\y\ y = F x) L K.
```

```
type mapfun    (A -> B) -> list A -> list B -> o.
type reducefun (A -> B -> B) -> list A -> B -> B -> o.

mapfun F nil nil.
mapfun F (X::L) ((F X)::K) :- mapfun F L K.

reducefun F nil Z Z.
reducefun F (H::T) Z (F H R) :- reducefun F T Z R.


?- mapfun F (a::b::nil) ((g a a)::(g a b)::nil).


(x\ g x x)    (x\ g a x)    (x\ g x a)    (x\ g a a)


?- mapfun F (a::b::nil) (c::d::nil).


?- reducefun (x\y\ x + y) (3::4::8::nil) 6 R.
R = 3 + (4 + (8 + 6))

?- reducefun F (4::8::nil) 6 (1 + (4 + (1 + (8 + 6)))).
F = x\y\ 1 + (4 + (1 + (8 + 6)));

F = x\y\ 1 + (x + (1 + (8 + 6)));

F = x\y\ 1 + (x + y);
no

?-


?- pi z\ reducefun F (4::8::nil) z (1 + (4 + (1 + (8 + z)))).
F = x\y\ 1 + (x + y);
no

?-
```

8.2 Functional difference lists

```
kind dlist        type -> type.
type dl           list A -> list A -> dlist A.

type concat       dlist A -> dlist A -> dlist A -> o.
concat (dl L1 L2) (dl L2 L3) (dl L1 L3).

kind btree        type -> type.
type empty        btree A.
type bt           A -> btree A -> btree A -> btree A.

type collect      btree A -> list A -> o.
type aux          btree A -> dlist A -> o.
collect Bt L :- aux Bt (dl L nil).
```

```
aux empty          (dl A A).
aux (bt N L R) (dl A B) :- aux L (dl A (N::C)), aux R (dl C B).

kind fdlist      type -> type.
type fdl         (list A -> list A) -> fdlist A.

type collect'    btree A -> list A -> o.
type aux'        btree A -> fdlist A -> o.
collect' Bt (A nil) :- aux' Bt (fdl A).
aux' empty       (fdl x\ x).
aux' (bt N L R) (fdl x\ A (N::(B x))) :- aux' L (fdl A), aux' R (fdl B).


type palindrome    fdlist A -> o.
palindrome (fdl x\x).
palindrome (fdl x\ Y::x).
palindrome (fdl x\ Y::(F (Y::x))) :- palindrome (fdl F).


?- palindrome (fdl x\ a::b::c::b::a::x).
solved

?- palindrome (fdl x\ a::b::a::(F x)).
F = x\ x;

F = x\ b::a::x;

F = x\ a::b::a::x;

F = x\ Y::a::b::a::x;

F = x\ Z::Z::a::b::a::x;

F = x\ Y::Z::Y::a::b::a::x;

F = x\ Y::Z::Z::Y::a::b::a::x;

F = x\ Y::Z::U::Z::Y::a::b::a::x;

F = x\ Y::Z::U::U::Z::Y::a::b::a::x

?-
```

9 Higher-order unification is not a panacea

```
kind i      type.
type a,b    i.
type f      i -> i -> i.


type extract_a   i -> (i -> i) -> o.
extract_a (F a) F.
```

```
?- extract_a (f a (f a b)) F.
F = x\ f x (f x b);

F = x\ f x (f a b);

F = x\ f a (f x b);

F = x\ f a (f a b);
no

?-


extract_a (F a) F :- !.


pi a\ sigma F\ (F a) = (f a (f a b)).


sigma F\ pi a\ (F a) = (f a (f a b)).


extract_a a x\x.
extract_a b x\b.
extract_a (f T S) (x\ f (U x) (V x)) :- extract_a T U, extract_a S V.


type rewrite   int -> int -> o.

rewrite (0 + X) X.
rewrite (1 * X) X.
rewrite (X - X) 0.
rewrite (C X) (C Y) :- rewrite X Y.
```

10 Comparison with functional programming

```
type eq_pred (A -> o) -> (A -> o) -> o.

eq_pred R R.


?- eq_pred (x\ p x, q x) (x\ q x, p x).
```

11 Bibliographic notes

Chapter 6 Mechanisms for Structuring Large Programs

1 Desiderata for modular programming

2 A modules language

```
   module <name>.
```

```
module smlists.

type id                 list A -> list A -> o.
type memb, member       A -> list A -> o.
type revapp             list A -> list A -> list A -> o.
type reverse            list A -> list A -> o.
type append             list A -> list A -> list A -> o.
type memb_and_rest      A -> list A -> list A -> o.

id nil nil.
id (X::L) (X::K) :- id L K.

memb X (X::L).
memb X (Y::L) :- memb X L.

member X (X::L) :- !.
member X (Y::L) :- member X L.

revapp nil L L.
revapp (X::L1) L2 L3 :- revapp L1 (X::L2) L3.

reverse L1 L2 :- revapp L1 nil L2.

append nil K K.
append (X::L) K (X::M) :- append L K M.

memb_and_rest X (X::L) L.
memb_and_rest X (Y::K) (Y::L) :- memb_and_rest X K L.

end


sig smlists.

type id                 list A -> list A -> o.
type memb, member       A -> list A -> o.
type reverse            list A -> list A -> o.
type append             list A -> list A -> list A -> o.
type memb_and_rest      A -> list A -> list A -> o.

end


   sig <name>.


[smlists] ?-


   accumulate <name1>, ..., <namen>.


sig smpairs.

kind pair    type -> type -> type.
```

```
type pr      A -> B -> pair A B.

type assoc, assod   A -> B -> list (pair A B) -> o.
type domain         list (pair A B) -> list A -> o.
type range          list (pair A B) -> list B -> o.

end

module smpairs.
accumulate smlists.

kind pair    type -> type -> type.
type pr      A -> B -> pair A B.

type assoc, assod   A -> B -> list (pair A B) -> o.

assoc X Y L :- memb (pr X Y) L.
assod X Y L :- member (pr X Y) L.

type domain         list (pair A B) -> list A -> o.

domain nil nil.
domain ((pr X Y)::Alist) (X::L) :- domain Alist L.

type range          list (pair A B) -> list B -> o.

range nil nil.
range ((pr X Y)::Alist) (Y::L)  :- range Alist L.
end


   type  append  list A -> list A -> list A -> o.


   accum_sig  <name1>, ..., <namen>.


   accum_sig smlists.
```

3 Matching signatures and modules

```
kind   item   type.
type   p,q    item -> o.
type   a      item.
type   r      list item -> o.


module m3'.
accum_sig m1, m2.
type   s,t   item -> o.
type   b     item.
s X :- p X.
t b.
end
```

```
kind item type.
type p,q,r,s,t  item -> o.
type b          item.


sig m1.
kind   item   type.
type   p,q    item -> o.
type   a      item.
type   r      list item -> o.
end.
```

4 The logical interpretation of modules

4.1 Existential quantification in program clauses

4.2 A module as a logical formula

4.3 Interpreting queries against modules

```
[m] ?- g X.


[m3] ?- p a.


[m3] ?- s a.


[m3] ?- sigma x\ t x.


[m3] ?- t X.
```

4.4 Module accumulation as scoped inlining of code

```
kind item type.
type p,q,r,r',s,t item -> o.
type a,b item.


p X :- q X.
r' (a :: nil).
q X :- r X.
r a.
s X :- p X.
t b.
```

5 Some programming aspects of the modules language

5.1 Hiding and abstract datatypes

```
sig stack.
kind  store          type -> type.
type  init           store A -> o.
type  add, remove    A -> store A -> store A -> o.
```

```
end

module stack.
kind  store         type -> type.
type  emp           store A.
type  stk           A -> store A -> store A.
type  init          store A -> o.
type  add, remove   A -> store A -> store A -> o.
init emp.
add    X S (stk X S).
remove X (stk X S) S.
end


[stack] ?- init A.


[stack] ?- sigma A\ sigma B\ sigma C\ init A, add 1 A B, remove X B C.


sig graph_search.
kind  action  type.
type  graph_search  list action -> o.
end

module graph_search.
accumulate stack.
kind state, action type.
type graph_search list action -> o.
type init_open store state -> o.
type expand_graph store state -> list state -> list action -> o.
...
graph_search Soln :- init_open Open, expand_graph Open nil Soln.
init_open Open :- start_state State, init Op, add State Op Open.
expand_graph Open Closed Soln :-
     remove State Open Rest, final_state State, soln State Soln.
expand_graph Open Closed Soln :-
     remove State Open ROpen,
     expand_node State NStates,
     add_states NStates ROpen (State::Closed) NOpen,
     expand_graph NOpen (State::Closed) Soln.
...
end
```

5.2 Code extensibility and modular composition

```
sig proplogic.
kind  form          type.
type  ff, tt        form.
type  and, or, ==>  form -> form -> form.
type  prove         list form -> form -> o.
end

module proplogic.
```

```
accumulate smlists.
kind  form          type.
type  ff, tt        form.
type  and, or, ==>  form -> form -> form.
type  prove         list form -> form -> o.
prove L F :- member ff L.
prove L F :-
   memb_and_rest (and A B) L L', prove (A::B::L') F.
prove L (and F1 F2) :- prove L F1, prove L F2.
prove L (==> F1 F2) :- prove (F1::L) F2.
prove L F :- memb_and_rest (or A B) L L',
             prove (A::L') F, prove (B::L') F.
prove L (or F1 F2) :- prove L F1; prove L F2.
prove L F :- member F L.
prove L F :- memb_and_rest (==> F1 F2) L L',
             prove L F1, prove (F2::L') F.
end

sig quantlogic.
accum_sig proplogic.
kind term        type.
type all, some   (term -> form) -> form.
end

module quantlogic.
accumulate proplogic, smlists.
kind term        type.
type all, some   (term -> form) -> form.

prove L F :- memb_and_rest (some P) L _,
             pi c\ prove ((P c)::L) F.
prove L (all P) :- pi c\ prove L (P c).
prove L (some P) :- prove L (P T).
prove L F :- memb_and_rest (all P) L K,
             append K [all P] M, prove ((P T)::M) F.
end


sig proplogic.
kind  form   type.
type  ff     form.
type  and    form -> form -> form.
type  or     form -> form -> form.
type  ==>    form -> form -> form.
type  prove  list form -> form -> o.
end

module proplogic.
accumulate smlists.
kind  form   type.
type  ff, tt form.
type  and    form -> form -> form.
type  or     form -> form -> form.
type  ==>    form -> form -> form.
```

```
type  prove  list form -> form -> o.
prove Gamma F :- member ff Gamma.
prove Gamma F :-
   memb_and_rest (and A B) Gamma Gamma', prove (A :: B :: Gamma') F.
prove Gamma (and F1 F2) :- prove Gamma F1, prove Gamma F2.
prove Gamma (==> F1 F2) :- prove (F1 :: Gamma) F2.
prove Gamma F :-
   memb_and_rest (or A B) Gamma Gamma',
   prove (A :: Gamma') F, prove (B :: Gamma') F.
prove Gamma (or F1 F2) :- prove Gamma F1; prove Gamma F2.
prove Gamma F :- member F Gamma.
prove Gamma F :-
   memb_and_rest (==> F1 F2) Gamma Gamma',
   prove Gamma F1, prove (F2 :: Gamma') F.
end


sig quantlogic.
accum_sig proplogic.
kind term type.
type all, some   (term -> form) -> form.
end

module quantlogic.
accumulate proplogic, smlists.
kind term type.
type all, some   (term -> form) -> form.

prove Gamma F :-
  memb_and_rest (some P) Gamma Gamma',
  pi c \ prove ((P c) :: Gamma) F.
prove Gamma (all P) :- pi c\ prove Gamma (P c).
prove Gamma (some P) :- prove Gamma (P T).
prove Gamma F :-
  memb_and_rest (all P) Gamma Gamma',
  append Gamma' [all P] Gamma'',
  prove ((P T) :: Gamma'') F.
end
```

5.3 Signature accumulation and parametrization of modules

```
sig mylogic.
accum_sig quantlogic.
type  a, b  term.
type  q  form.
type  p  term -> form.
end

module mylogic.
accumulate  quantlogic, smlists.
type  a, b  term.
type  q  form.
type  p  term -> form.
```

```
end
```

5.4 Higher-order programming and predicate visibility

```
[test] ?- test X.
```

```
sig comblibrary.
type  call  o -> o.
end
```

```
module comblibrary.
type  call  o -> o.
type  p  list int -> o.
p (1 :: nil).
call Q :- Q.
end
```

```
sig test.
type  test  list int -> o.
end
```

```
module test.
accumulate comblibrary.
type  test  list int -> o.
type  p  list int -> o.
p (2 :: nil).
test X :- call (p X).
end
```

```
type  test  list int -> o.
type  call  o -> o.
type  p, p'  list int -> o.
p' (1:: nil).
p (2 :: nil).
call P :- P.
test X :- call (p X).
```

6 Implementation considerations

7 Bibliographic notes

Chapter 7 Computations over λ-Terms

1 Representing objects with binding structure

```
  static List[] empty (int n) {
    List[] g;
    int i;
    g = new List [n];
    for (i=0; i <= n-1; i = i+1) {g[i] = null;}
    return(g); }
```

```
fun fib 0 = 0 |
    fib n =
      let fun iter(this,prev,count) =
        if count = n then this
        else iter(this+prev,this,count+1)
      in iter(1,0,1)
```

```
        end;
```

1.1 Encoding logical formulas with quantifiers

```
type    ff,                             % encoding the false proposition
        tt  form.                       % encoding the true proposition
type    &&,                             % encoding conjunction
        !!,                             % encoding disjunction
        ==>   form -> form -> form.     % encoding implication
type    neg   form -> form.             % encoding negation
infixl  &&  5.
infixl  !!  4.
infixr  ==> 3.


type   p   term -> form.
type   q   term -> term -> form.
type   f   term -> term.
type   a   term.


 x\ (p x) ==> (p (f x))    Z\ (p Z) ==> (p (f Z))     Z\ (p Z) && (p (f Z))


type  all, some      (term -> form) -> form.


   all x\ (p x) ==> (p (f x))       some Z\ (p Z) && (p (f Z))


all x\ all y\ (q x y) ==> some z\ p z && q z y.
```

1.2 Encoding untyped λ-terms

```
kind   tm     type.
type   app    tm -> tm -> tm.
type   abs    (tm -> tm) -> tm.
```

1.3 Properties of the encoding of binding

2 Realizing object-level substitution

```
all x\ p x ==> some y\ q x y


(x\ p x ==> some y\ q x y) (f a)


p (h a) ==> some y\ q (f a) y.


type list_instan     list term -> form -> form -> o.
list_instan nil B B.
```

```
list_instan (T::Ts) (all B) C :- list_instan Ts (B T) C.


?- prog P, interp P (path a X).


type interp      form -> form -> o.
type backchain   form -> form -> form -> o.
type atom        form -> o.

interp D tt.
interp D (G1 && G2)  :- interp D G1, interp D G2.
interp D (G1 !! G2)  :- interp D G1; interp D G2.
interp D (some   G)  :- interp D (G X).
interp D A           :- atom A, backchain D D A.
backchain D A A.
backchain D (D1 && D2) A   :- backchain D D1 A; backchain D D2 A.
backchain D (all D1) A     :- backchain D (D1 X) A.
backchain D (G ==> D1) A   :- backchain D D1 A, interp D G.


type a, b, c    term.
type adj, path  term -> term -> form.
type prog       form -> o.

atom (adj _ _) & atom (path _ _).
prog ((adj a b) && (adj b c) &&
      (all x\ all y\ (adj x y) ==> (path x y)) &&
      (all x\ all y\ all z\  (adj x y) && (path y z) ==> (path x z))).


?- cbn (app (abs x\ abs w\w) (app (abs x\ app x x) (abs x\ app x x))) V.


?- cbv (app (abs x\ abs w\w) (app (abs x\ app x x) (abs x\ app x x))) V.


type cbn, cbv   tm -> tm -> o.

cbn (abs R) (abs R).
cbn (app M N) V :- cbn M (abs R), cbn (R N) V.

cbv (abs R) (abs R).
cbv (app M N) V :- cbv M (abs R), cbv N U, cbv (R U) V.
```

3 Mobility of binders

```
type  term   tm -> o.
term T.


type  app   tm -> (tm -> tm).
type  abs   (tm -> tm) -> tm.
```

```
pi M\ term M => pi N\ term N => term (app M N).
pi R\ (pi x\ term x => term (R x)) => term (abs R).


term (app M N) :- term M, term N.
term (abs R)   :- pi x\ term x => term (R x).


?- (term (abs y\ app y y)).


?- pi x\ term x => term ((y\ app y y) x).


?- pi x\ term x => term (app x x).


?- term (app c c).
```

4 Computing with untyped λ-terms

4.1 Computing normal forms

```
type bnorm, bbnorm  tm -> o.

bnorm (abs M)    :- pi x\ bbnorm x => bnorm (M x).
bnorm H          :- bbnorm H.
bbnorm (app M N) :- bbnorm M, bnorm N.


type redex, red1, reduce  tm -> tm -> o.

redex (app (abs R) N) (R N).
red1 M N :- redex M N.
red1 (app M N) (app P N) & red1 (app N M) (app N P) :- red1 M P.
red1 (abs M) (abs N) :- pi x\ red1 (M x) (N x).

reduce M M :- bnorm M.
reduce M N :- red1 M P, reduce P N.

type repeat    (A -> A -> o) -> A -> A -> o.
type reduce'   tm -> tm -> o.

repeat Pred M N :- Pred M P, !, repeat Pred P N.
repeat Pred M M.

reduce' M N :- repeat red1 M N.


redex (abs x\ app M x) M.
```

4.2 Reduction based on paths through terms

```
kind path type.
type bnd           (path -> path) -> path.
```

```
type left, right   path -> path.
type path          tm -> path -> o.

path (app M _) (left P) &
path (app _ M) (right P) :- path M P.
path (abs R) (bnd S) :- pi x\ pi p\ path x p => path (R x) (S p).


bnd u\ left u
bnd u\ right (bnd v\ left v)
bnd u\ right (bnd v\ right u)


?- foreach (path N)
           ((bnd u\ left u) ::
            (bnd u\ right (bnd v\ left v))::
            (bnd u\ right (bnd v\ right u))::nil).
N = abs W1\ app W1 (abs W2\ app W2 W1);
no

?-


type beta               tm -> (tm -> tm) -> tm.
type addbeta            tm -> tm -> o.
type bpath              tm -> path -> o.

bpath (app M _) (left P)  &
bpath (app _ M) (right P) :- bpath M P.
bpath (abs R)   (bnd S)   :- pi x\ pi p\ bpath x p =>
                                    bpath (R x) (S p).
bpath (beta N R) P :-
   pi x\ (pi Q\ bpath x Q :- bpath N Q) => bpath (R x) P.

addbeta (app (abs R) N) (beta M S) :- addbeta (abs R) (abs S),
                                      addbeta N M.
addbeta (app (app M N) P) (app O Q) :- addbeta (app M N) O,
                                       addbeta P Q.
addbeta (abs R) (abs S) :-
  pi x\ (pi M\ pi N\ addbeta (app x M) (app x N) :- addbeta M N) =>
        (addbeta x x) => addbeta (R x) (S x).


?- sigma B\ addbeta (app (abs x\x) (abs x\x)) B, bpath B Path.
Path = bnd W1\ W1;
no

?- foreach (P\ path T P) (bnd (W1\ W1) :: nil).
T = abs W1\ W1;
no

?-


?- sigma K\ sigma S\ sigma B\  K = (abs x\ abs y\ x),
```

```
   S = (abs x\ abs y\ abs z\ app (app x z) (app y z)),
   addbeta (app K (app S K)) B, bpath B Path.


?- foreach (path T)
    ((bnd W1\ bnd W2\ bnd W3\ left (left (bnd W4\ bnd W5\ W4))::
     (bnd W1\ bnd W2\ bnd W3\ left (right W3))::
     (bnd W1\ bnd W2\ bnd W3\ right (left W2))::
     (bnd W1\ bnd W2\ bnd W3\ right (right W3))::nil).
T = abs W1\ abs W2\ abs W3\ app (app (abs W4\ abs W5\ W4) W3) (app W2 W3);
no

?-
```

4.3 Type inference

```
kind ty type.
type arr      ty -> ty -> ty.
type typeof   tm -> ty -> o.

typeof (app M N) A :- typeof M (arr B A), typeof N B.
typeof (abs M) (arr A B) :- pi x\ typeof x A => typeof (M x) B.


?- typeof (abs x\ abs y\ abs z\ app (app x z) (app y z)) Ty.
Ty = arr (arr T1 (arr T2 T3)) (arr (arr T1 T2) (arr T1 T3))

?- typeof (abs x\x) Ty.
Ty = arr T1 T1

?- typeof (abs x\ app x x) Ty.
no

?-


?- typeof (abs x\x) (arr i i).
yes

?- typeof (abs x\x) (arr i Ty).
Ty = i

?-
```

4.4 Translating to and from de Bruijn syntax

```
kind deb    type.
type ab     deb -> deb.
type ap     deb -> deb -> deb.
type deb    int -> deb.


type trans    int -> tm -> deb -> o.
```

```
type depth    int -> tm -> o.

trans D (abs M)   (ab P)   :- pi c\ depth D c =>
                                  (E is D + 1, trans E (M c) P).
trans D (app M N) (ap P Q) :- trans D M P, trans D N Q.
trans D X         (deb E)  :- depth N X, E is (D - N).


?- trans 1 (abs x\ app x (abs y\ app x (abs w\ app w x))) D.
D = ab (ap (deb 1) (ab (ap (deb 2) (ab (ap (deb 1) (deb 3))))))

?- trans 1 P (ab (ap (deb 1) (ab (ap (deb 2) (ab (ap (deb 1) (deb 3)))))))).
P = abs W1\ app W1 (abs W2\ app W1 (abs W3\ app W3 W1))

?-


?- trans 1 (abs x\ abs y\ abs z\ y) P.


?- trans 2 (abs y\ abs z\ y) P1.


?- trans 3 (abs z\ d) P2.


?- trans 4 d P3.


?- depth N d, E is (4 - N).


trans D (app M N) (ap P Q) :- trans D M P, trans D N Q.
trans D (abs M)   (ab P)   :-  pi u\
    (pi N\ pi H\ trans N u (deb H) :- H is (N - D))
                               => (D' is D + 1, trans D' (M u) P).


?- trans 1 (abs x\ abs y\ abs z\ y) P.


pi N\ pi H\ trans N c (deb H) :- H is (N - 1).
pi N\ pi H\ trans N d (deb H) :- H is (N - 2).
pi N\ pi H\ trans N e (deb H) :- H is (N - 3).


?- trans 4 d P3.
```

5 Computations over first-order formulas

```
type vacuous        form -> form -> o.
type quantfree      form -> o.

vacuous (all  x\ P) P.
vacuous (some x\ P) P.
```

```
quantfree tt & quantfree ff.
quantfree F :- atom F.
quantfree (neg F) :- quantfree F.
quantfree (F && G) & quantfree (F !! G) & quantfree (F ==> G) :-
  quantfree F, quantfree G.


type nnf   form -> form -> o.

nnf A A & nnf (neg A) (neg A) :- atom A.
nnf (neg (neg B)) D           :- nnf B D.
nnf (neg (B && C)) (D !! E)   &
nnf (neg (B !! C)) (D && E)   :- nnf (neg B) D, nnf (neg C) E.
nnf (B ==> C) (D !! E)        :- nnf (neg B) D, nnf C E.
nnf (B !! C)  (D !! E)        &
nnf (B && C)  (D && E)        :- nnf B D, nnf C E.

nnf (neg (all B)) (some D) &
nnf (neg (some B)) (all D) :- pi x\ nnf (neg (B x)) (D x).
nnf (all B)  (all D)       &
nnf (some B) (some D)      :- pi x\ nnf (B x) (D x).


(all x\ (p x) && (all y\ q x y) && (p (f x))) !! (p a)


all x\ all y\ ((p x) && (q x y) && (p (f x))) !! (p a).


?- prenex ((all x\ q x x) && (all z\ all y\ q z y)) P.


all z\ all y\ (q z z) && (q z y)
all x\ all z\ all y\ (q x x) && (q z y)
all z\ all x\ (q x x) && (q z x)
all z\ all x\ all y\ (q x x) && (q z y)
all z\ all y\ all x\ (q x x) && (q z y)


type  prenex, merge       form -> form -> o.

prenex A A & prenex (neg A)  (neg A)  :- atom A.
prenex (B && C) D :- prenex B U, prenex C V,
                     merge (U && V) D.
prenex (B !! C) D :- prenex B U, prenex C V,
                     merge (U !! V) D.
prenex (all B)  (all D)  &
prenex (some B) (some D) :- pi x\ prenex (B x) (D x).

merge ((all  B) && (all C)) (all D) :-
  pi x\ merge ((B x) && (C x)) (D x).
merge ((all  B) && C) (all D)  &
merge ((some B) && C) (some D) :-
  pi x\ merge ((B x) && C) (D x).
merge (B && (all  C)) (all D)  &
merge (B && (some C)) (some D) :-
```

```
  pi x\ merge (B && (C x)) (D x).
merge ((some B) !! (some C)) (some D) :-
  pi x\ merge ((B x) !! (C x)) (D x).
merge ((some B) !! C) (some D) &
merge ((all B) !! C)  (all D)  :-
  pi x\ merge ((B x) !! C) (D x).
merge (B !! (some C)) (some D) &
merge (B !! (all C))  (all D)  :-
   pi x\ merge (B !! (C x)) (D x).

merge B B :- quantfree B.


type fohcG, fohcD, fohhG, fohhD   form -> o.

fohcG tt.
fohcG A           :- atom A.
fohcG (some B)    :- pi x\ fohcG (B x).
fohcG (B && C) & fohcG (B !! C) :- fohcG B, fohcG C.

fohcD A           :- atom A.
fohcD (G ==> D)   :- fohcG G,  fohcD D.
fohcD (D1 && D2)  :- fohcD D1, fohcD D2.
fohcD (all D)     :- pi x\ fohcD (D x).

fohhG tt.
fohhG A         :- atom A.
fohhG (D ==> G) :- fohhD D, fohhG G.
fohhG (B && C) & fohhG (B !! C) :- fohhG B, fohhG C.
fohhG (some B) & fohhG (all B)  :- pi x\ fohhG (B x).

fohhD A           :- atom A.
fohhD (D1 && D2)  :- fohhD D1, fohhD D2.
fohhD (G ==> D)   :- fohhG G,  fohhD D.
fohhD (all D)     :- pi x\ fohhD (D x).


interp D (D1 ==> G) :- interp (D1 && D) G.
interp D (all G) :- pi x\ interp D (G x).
```

6 Specifying object-level substitution

```
type  subst (tm -> tm) -> tm -> tm -> o.
subst R N (R N).


?- copy (abs x\ abs y\ app y x) M.


?- copy (abs y\ app y c) (M1 c).


?- copy (app d c) (M2 d).
```

```
 M2 = u\ app u c    M1 = v\ abs u\ app u v    M = abs v\ abs u\ app u v.


type copy  tm -> tm -> o.
type subst (tm -> tm) -> tm -> tm -> o.

copy (app M N) (app P Q) :- copy M P, copy N Q.
copy (abs M)   (abs N)   :- pi x\ copy x x => copy (M x) (N x).

subst M T S :- pi x\ copy x T => copy (M x) S.


pi x\ copy x T => copy (M x) S


kind  term, form  type.

type  copyterm    term -> term -> o.
type  copyform    form -> form -> o.
type  subst       (term -> form) -> term -> form -> o.

copyterm a       a.
copyterm (f X)   (f U)       :- copyterm X U.
copyterm (g X Y) (g U V)     :- copyterm X U, copyterm Y V.

copyform tt tt.
copyform ff ff.
copyform (neg B)   (neg D)    :- copyform B D.
copyform (B && C)  (D && E)   &
copyform (B !! C)  (D !! E)   &
copyform (B ==> C) (D ==> E)  :- copyform B D, copyform C E.
copyform (all B)   (all D)    &
copyform (some B)  (some D) :- pi y\ copyterm y y => copyform (B y) (D y).

copyform (p X)   (p U)       :- copyterm X U.
copyform (q X Y) (q U V)     :- copyterm X U, copyterm Y V.

subst M T N :- pi x\ copyterm x T => copyform (M x) N.


type  a  term.
type  f  term -> term.
type  g  term -> term -> term.
type  p  term -> form.
type  q  term -> term -> form.


type subst      (term -> form) -> term -> form -> o.
type substterm  (term -> term) -> term -> term -> o.

subst (x\ tt) T tt.
subst (x\ ff) T ff.
subst (x\ neg (B x))       T (neg D)   :- subst B T D.
subst (x\ (B x) && (C x))  T (D && E)  &
subst (x\ (B x) !!  (C x)) T (D !! E)  &
```

```
subst (x\ (B x) ==> (C x)) T (D ==> E) :- subst B T D, subst C T E.
subst (x\ all  (B x))      T (all  D)  &
subst (x\ some (B x))      T (some D)  :-
   pi y\ substterm (x\y) T y => subst (x\ B x y) T (D y).

subst (x\ p (X x))       T (p U)    :- substterm X T U.
subst (x\ q (X x) (Y x)) T (q U V)  :- substterm X T U, substterm Y T V.

substterm (x\ x) T T.
substterm (x\ a) T a.
substterm (x\ f (F x)) T (f S) :- substterm F T S.
substterm (x\ g (F x) (G x)) T (g S R) :-  substterm F T S, substterm G T R.


?- subst M (f a) ((p (f a)) && (q a (f a))).


M = x\ (p x) && (q a x)
M = x\ (p x) && (q a (f a))
M = x\ (p (f a)) && (q a x)
M = x\ (p (f a)) && (q a (f a))
```

7 The λ-tree approach to abstract syntax

8 The L_λ subset of λProlog

```
redex (app (abs R) N) (R N).


redex (app (abs R) N) S :- subst R N S.


?-  M = N.


?-  copy M N.


kind i  type.
type a  i.
type f  i -> i.
type copyi  i -> i -> o.

copyi a a.
copyi (f X) (f Y) :- copyi X Y.


?-  copyi M N.
```

9 Bibliographic notes

Chapter 8 Unification of λ-Terms

1 Properties of the higher-order unification problem

2 A procedure for checking for unifiability

3 Higher-order pattern unification

4 Pragmatic aspects of higher-order unification

```
type  subst (tm -> tm) -> tm -> tm -> o.
subst R N (R N).
```

5 Bibliographic notes

Chapter 9 Implementing Proof Systems

1 Deduction in propositional intuitionistic logic

```
kind   term, form  type.                % types for terms and formulas

type    ff,                             % encoding the false proposition
        tt    form.                     % encoding the true proposition
type    &&,                             % encoding conjunction
        !!,                             % encoding disjunction
        ==>   form -> form -> form.     % encoding implication

infixl  &&  5.
infixl  !!  4.
infixr  ==> 3.


(a ==> (a ==> b) ==> (a ==> b ==> c) ==> c


type a, b, c   form.
atom a & atom b & atom c.


?- pi a\ pi b\ pi c\ atom a => atom b => atom c =>
                     seq nil (a ==> (a ==> b) ==> (a ==> b ==> c) ==> c).


type  atom     form -> o.
type  seq      list form -> form -> o.

seq Gamma A :- atom A, memb_and_rest A Gamma _.
seq Gamma tt.
seq Gamma (A && B)  :- seq Gamma A, seq Gamma B.
seq Gamma (A !! B)  &
seq Gamma (B !! A)  :- seq Gamma A.
seq Gamma (A ==> B) :- seq (A::Gamma) B.
seq Gamma _ :- memb_and_rest ff Gamma _.
seq Gamma G :- memb_and_rest (A && B) Gamma Gamma', seq (A::B::Gamma') G.
seq Gamma G :- memb_and_rest (A !! B) Gamma Gamma',
               (seq (A::Gamma') G ; seq (B::Gamma') G).
seq Gamma G :- memb_and_rest (A ==> B) Gamma Gamma',
  ( atom A, memb_and_rest A Gamma' _, seq (B::Gamma') G ;
    A = (C && D),  seq ((C ==> D ==> B) :: Gamma') G ;
    A = (C !! D),  seq ((C ==> B) :: (D ==> B) :: Gamma') G ;
```

```
    A = (C ==> D), seq ((D ==> B) :: Gamma') A, seq (B :: Gamma') G ;
    A = ff,        seq Gamma' G;
    A = tt,        seq (B :: Gamma') G  ).
```

2 Encoding natural deduction for intuitionistic logic

```
type  all, some      (term -> form) -> form.


kind  proof          type.

type true_i          proof.
type false_e         form  -> proof -> proof.
type and_i           proof -> proof -> proof.
type and_e1, and_e2  form  -> proof -> proof.
type imp_i           (proof -> proof) -> proof.
type imp_e           form -> proof -> proof -> proof.
type or_i1, or_i2    proof -> proof.
type or_e            form  -> form -> proof ->
                     (proof -> proof) -> (proof -> proof) -> proof.
type all_e           term  -> (term -> form) -> proof -> proof.
type all_i           (term -> proof) -> proof.
type some_e          (term -> form) -> proof ->
                     (term -> proof -> proof) -> proof.
type some_i          term  -> proof -> proof.


type  #              proof -> form -> o.
infix # 2.

true_i # tt.
(and_i P1 P2) # (A && B) :- (P1 # A), (P2 # B).
(or_i1 P) # (A !! B)     :- P # A.
(or_i2 P) # (A !! B)     :- P # B.
(imp_i Q) # (A ==> B)    :- pi p\ (p # A) => ((Q p) # B).
(some_i T P) # (some A)  :- P # (A T).
(all_i Q) # (all A)      :- pi y\ (Q y) # (A y).
(false_e A P) # A        :- P # ff.
(and_e1 B P) # A         :- P # (A && B).
(and_e2 A P) # B         :- P # (A && B).
(or_e A B P Q1 Q2) # C    :- (P # (A !! B)),
                            (pi p1\ (p1 # A) => ((Q1 p1) # C)),
                            (pi p2\ (p2 # B) => ((Q2 p2) # C)).
(imp_e A P1 P2) # B   :- (P1 # A), (P2 # (A ==> B)).
(some_e A P1 Q) # B   :- (P1 # (some A)),
                         pi y\ pi p\ (p # (A y)) => ((Q y p) # B).
(all_e T A P) # (A T) :- (P # (all A)).


type a, b      form.          % propositional constants
type q         term -> form.  % a predicate of one argument
type c         term.          % a first-order constant
type f         term -> term.  % a term constructor
```

```
?- (imp_i w\w) # (a ==> a).

?- (imp_i x\ imp_i y\ imp_e a x y) # (a ==> ((a ==> b) ==> b)).

?- (imp_i P\ all_i y\ imp_i Q\
     (imp_e (q (f y))
         (imp_e (q y) Q (all_e y (x\ (q x) ==> (q (f x))) P))
         (all_e (f y) (x\ (q x) ==> (q (f x))) P)))
    # ((all x\ (q x) ==> (q (f x))) ==>
       (all x\ (q x) ==> (q (f (f x))))).


?- (imp_i w\ (and_i (and_e2 a w) (and_e1 b w))) # R.
R = a && b ==> b && a


?- (imp_i P\ all_i y\ imp_i Q\
     (imp_e (q (f y)) (imp_e (q y) Q (all_e y A P))
            (all_e (f y) A' P))) # B.
A = w\ p w ==> p (f w)
A' = w\ p w ==> p (f w)
B = all (w\ p w ==> p (f w)) ==> all (w\ p w ==> p (f (f w)))

?-


?- P # (a ==> ((a ==> b) ==> b)).
```

3 A theorem-prover for classical logic

```
kind  atm      type.
type  p, n     atm -> form.


kind pair   type -> type -> type.
type pr     A -> B -> pair A B.

kind que    type -> type.
type que    (list (pair A int) -> list (pair A int)) -> que A.
type qpush  A -> que A -> que A -> o.
type qpop   A -> que A -> que A -> o.
type bound  int -> o.

qpush X (que k\ Phi k) (que k\ Phi ((pr X N)::k)) :- bound N.
qpop  X (que k\ ((pr X 1)::(Phi k)))
        (que k\ Phi k).
qpop  X (que k\ ((pr X N)::(Phi k)))
        (que k\ Phi ((pr X M)::k)) :- N > 1, M is N - 1.


type lit         form -> o.
type prv         list form -> que form -> o.

prv (tt       :: Gamma) Phi.
```

```
prv ((B && C) :: Gamma) Phi :- prv (B::Gamma) Phi,
                               prv (C::Gamma) Phi.
prv ((B !! C) :: Gamma) Phi :- prv (B::C::Gamma) Phi.
prv ((all B)  :: Gamma) Phi :- pi x\ prv ((B x)::Gamma) Phi.
prv ((some B) :: Gamma) Phi :- qpush (some B) Phi Phi',
                               prv Gamma Phi'.
prv (p A      :: Gamma) Phi :- lit (p A) => prv Gamma Phi.
prv (n A      :: Gamma) Phi :- lit (n A) => prv Gamma Phi.
prv nil Phi :- lit (n A), lit (p A).
prv nil Phi :- qpop (some B) Phi Phi', prv ((B T)::nil) Phi'.


posints 1.
posints N :- posints M, N is M + 1.

thm B :- posints N, bound N => prv (B::nil) (que x\x).


prv (p A :: Gamma) Phi :- lit (n A) ; lit (p A) => prv Gamma Phi.
prv (n A :: Gamma) Phi :- lit (p A) ; lit (n A) => prv Gamma Phi.


prv (p A :: Gamma) Phi :- lit (n A), ! ; lit (p A) => prv Gamma Phi.
prv (n A :: Gamma) Phi :- lit (p A), ! ; lit (n A) => prv Gamma Phi.


((p (r c)) !! (p (r t)) !! (p (g c)) !! (some x\ (n (r x)) && (n (g x)))).
```

4 A general architecture for theorem provers

4.1 Goals and tactics

```
kind  goal   type.
type  trueg  goal.                  % vacuously true goal
type  cc     goal -> goal -> goal.  % conjunctive goal
type  allg   (A -> goal) -> goal.   % universally quantified goal
infixl cc    3.

type  goalreduce, redex, red1       goal -> goal -> o.
type  primgoal                      goal -> o.

redex (trueg cc G) G & redex (G cc trueg) G.
redex (allg x\trueg) trueg.

red1 G H :- redex G H, !.
red1 (G cc H) (Gx cc H) &
red1 (H cc G) (H cc Gx) :- red1 G Gx.
red1 (allg G) (allg Gx) :- pi x\ red1 (G x) (Gx x).

goalreduce G H :- red1 G Gx, !, goalreduce Gx H.
goalreduce G G.


kind node             type.
type a, b, c, d, e    node.
```

```
type adj, path        node -> node -> goal.

primgoal (adj _ _)  &  primgoal (path _ _).

type adj_tac, path_base_tac, path_rec_tac    goal -> goal -> o.

adj_tac (adj a b) trueg  &  adj_tac (adj a c) trueg.
adj_tac (adj b d) trueg  &  adj_tac (adj c d) trueg.
adj_tac (adj d a) trueg  &  adj_tac (adj d e) trueg.
path_base_tac  (path X Y) (adj X Y).
path_rec_tac   (path X Y) ((adj X Z) cc (path Z Y)).


type sq               list form -> form -> goal.
primgoal (sq _ _).

type initial, and_r, imp_r, all_r, and_l, imp_l, all_l, all_l'
                      goal -> goal -> o.

initial (sq Gamma A) trueg :- memb_and_rest A Gamma _.
and_r   (sq Gamma (A &&  B)) ((sq Gamma A)  cc (sq Gamma B)).
imp_r   (sq Gamma (A ==> B)) (sq (A::Gamma) B).
all_r   (sq Gamma (all A)) (allg x\ sq Gamma (A x)).
and_l   (sq Gamma A) (sq (B::C::Gamma') A) :-
              memb_and_rest (B && C) Gamma Gamma'.
imp_l   (sq Gamma A) ((sq Gamma B) cc (sq (C::Gamma') A)) :-
              memb_and_rest (B ==> C) Gamma Gamma'.
all_l   (sq Gamma A) (sq ((B T)::Gamma) A) :-
              memb_and_rest (all B) Gamma Gamma'.
all_l'  (sq Gamma A) (sq ((B T)::Gamma') A) :-
              memb_and_rest (all B) Gamma Gamma'.
```

4.2 Combining tactics into proof strategies

```
then Tac1 Tac2 In Out :- Tac1 In Mid, Tac2 Mid Out.


type maptac           (goal -> goal -> o) -> goal -> goal -> o.

maptac Tac trueg trueg.
maptac Tac (I1 cc I2) (O1 cc O2) :- maptac Tac I1 O1,
                                    maptac Tac I2 O2.
maptac Tac (allg In) (allg Out) :- pi t\ maptac Tac (In t) (Out t).
maptac Tac In Out :- primgoal In, Tac In Out.

type idtac           goal -> goal -> o.
type repeat, try     (goal -> goal -> o) -> goal -> goal -> o.
type then,
     orelse, orelse! (goal -> goal -> o) ->
                        (goal -> goal -> o) -> goal -> goal -> o.

idtac              In In.
```

```
then     Tac1 Tac2 In Out :- Tac1 In Mid, maptac Tac2 Mid Out.
orelse   Tac1 Tac2 In Out :- Tac1 In Out  ; Tac2 In Out.
orelse!  Tac1 Tac2 In Out :- Tac1 In Out, ! ; Tac2 In Out.
repeat   Tac       In Out :- orelse (then Tac (repeat Tac))
                                    idtac In Out.
try      Tac       In Out :- orelse Tac idtac In Out.


invertible In Out :-
    repeat (orelse and_r (orelse and_l (orelse imp_r all_r))) In Out.


?- invertible (sq [] ((a && (a ==> b)) ==> (a && b))) Out.
Out = ((sq (a :: (a ==> b) :: nil) a) cc (sq (a :: (a ==> b) :: nil) b))

?- invertible (sq [] ((all x\ (p x) ==> (p (f x))) ==>
                      (all x\ (p x) ==> (p (f (f x))))))
              Out.
Out = (allg (w\ sq (p w :: (all w\ p w ==> p (f w)) :: nil)
                   (p (f (f w)))))

?-


ip_decide (sq Gamma A) trueg :- seq Gamma A.


?- then invertible (then all_l (then all_l' ip_decide))
      (sq [] ((all x\ (p x) ==> (p (f x))) ==>
              (all x\ (p x) ==> (p (f (f x))))))
       Out.
Out = allg W1\ trueg

?-


then invertible (then all_l (then all_l' ip_decide))
```

5 Bibliographic notes

Chapter 10 Computations over Functional Programs

1 The miniFP programming language

```
(cns (cns (i 4) null) (cns (i 5) null))


kind tm       type.

% Lambda calculus with special forms
type abs      (tm -> tm) -> tm.          % function abstraction
type @        tm -> tm -> tm.            % application
infixl        @ 4.                       % application is infix
type cond     tm -> tm -> tm -> tm.      % conditional
type fixpt    (tm -> tm) -> tm.          % recursive functions
```

```
type cns      tm -> tm -> tm.            % list constructor

% Builtin datatypes and builtin functions over them
type i                            int -> tm. % integer coercion
type and, or, ff, tt                     tm. % for booleans
type cons, car, cdr, nullp, consp, null  tm. % for lists
type greater, zerop, minus, sum, times   tm. % for integers
type equal                               tm. % general equality


type prog     string -> tm -> o.
prog "fib" (fixpt fib\ abs n\
  cond (zerop @ n) (i 0)
       (cond (equal @ n @ (i 1)) (i 1)
             (sum @ (fib @ (minus @ n @ (i 1))) @
                    (fib @ (minus @ n @ (i 2)))))).

prog "mem" (fixpt mem\ abs x\ abs l\
  cond (nullp @ l) ff
       (cond (and @ (consp @ l) @ (equal @ (car @ l) @ x)) tt
             (mem @ x @ (cdr @ l)))).

prog "appnd" (fixpt appnd\ abs l\ abs k\
  cond (nullp @ l) k (cons @ (car @ l) @ (appnd @ (cdr @ l) @ k))).

prog "map" (fixpt map\ abs f\ abs l\
  cond (nullp @ l) null (cons @ (f @ (car @ l)) @
                                (map @ f @ (cdr @ l)))).


typeof (abs w\ w) (arr int int).
typeof (abs w\ w) (arr bool bool).
pi t\ typeof (abs w\ w) (arr t t).


?- sigma Exp\ prog Name Exp, typeof Exp Ty.


Ty = arr int int
Ty = arr A (arr (lst A) bool)
Ty = arr (lst A) (arr (lst A) (lst A))
Ty = arr (arr A B) (arr (lst A) (lst B))


kind ty               type.
type int, bool        ty.
type lst              ty -> ty.
type arr              ty -> ty -> ty.
type typeof           tm -> ty -> o.

typeof (M @ N) A          :- typeof M (arr B A), typeof N B.
typeof (cond P Q R) A     :- typeof P bool, typeof Q A, typeof R A.
typeof (abs M) (arr A B)  :- pi x\ typeof x A => typeof (M x) B.
typeof (fixpt M) A        :- pi x\ typeof x A => typeof (M x) A.
```

```
typeof tt bool & typeof and (arr bool (arr bool bool)).
typeof ff bool & typeof or  (arr bool (arr bool bool)).
typeof equal (arr A (arr A bool)).

typeof null  (lst A)             & typeof cons  (arr A (arr (lst A) (lst A))).
typeof car   (arr (lst A) A)     & typeof cdr   (arr (lst A) (lst A)).
typeof consp (arr (lst A) bool)  & typeof nullp (arr (lst A) bool).

typeof (i I) int                          & typeof zerop   (arr int bool).
typeof greater (arr int (arr int bool)) & typeof minus   (arr int (arr int int)).
typeof sum     (arr int (arr int int))  & typeof times   (arr int (arr int int)).
```

2 Specifying evaluation for miniFP programs

```
type eval        tm -> tm -> o.
type val         tm -> o.
type apply       tm -> tm -> tm -> o.
type eval_spec   tm -> list tm -> tm -> o.
type special     int -> tm -> o.

type spec        int -> tm -> list tm -> tm. % for specials

% Description of which expressions denote values
val (abs _) & val (i _)  & val tt   & val ff  & val null.
val (cns _ _) & val (spec _ _ _).

% eval and apply are the heart of evaluation
eval V V               :- val V.
eval (M @ N) V         :- eval M F, eval N U, apply F U V.
eval (fixpt R)    V    :- eval (R (fixpt R)) V.
eval (cond C L R) V    :- eval C B, if (B = tt) (eval L V)
                                                (eval R V).
eval F (spec I F nil)  :- special I F.

apply (abs R) U V :- eval (R U) V.
apply (spec 1 F Args) U V :- eval_spec F (U::Args) V.
apply (spec C F Args) U (spec D F (U::Args)) :- C > 1, D is C - 1.

% Declaration of the arity of the built-in functions
special 2 or    & special 2 and   & special 2 equal &
special 1 car   & special 1 cdr   & special 2 cons  &
special 1 nullp & special 1 consp & special 1 zerop &
special 2 minus & special 2 sum   & special 2 times &
special 2 greater.

% Description of how to compute the built-in functions
eval_spec car   ((cns V U)::nil) V.
eval_spec cdr   ((cns V U)::nil) U.
eval_spec cons  (U::V::nil) (cns V U).
eval_spec nullp (U::nil) V :- if (U = null) (V = tt) (V = ff).
eval_spec consp (U::nil) V :- if (U = null) (V = ff) (V = tt).
eval_spec and (C::B::nil) V :- if (B = ff) (V = ff)
```

```
                                    (if (C = ff) (V = ff) (V = tt)).
eval_spec or  (C::B::nil) V :- if (B = tt) (V = tt)
                                    (if (C = tt) (V = tt) (V = ff)).
eval_spec minus ((i N)::(i M)::nil) (i V) :- V is M - N.
eval_spec sum   ((i N)::(i M)::nil) (i V) :- V is M + N.
eval_spec times ((i N)::(i M)::nil) (i V) :- V is M * N.
eval_spec zerop ((i N)::nil) V  :- if (N = 0) (V = tt) (V = ff).
eval_spec equal (C::B::nil) V   :- if (B = C) (V = tt) (V = ff).
eval_spec greater ((i N)::(i M)::nil) V :-
```

2.1 A big-step style specification

```
?- prog "fib" F, eval (F @ (i 12)) V.


?- prog "fib" Fib, prog "map" Map,
   eval (Map @ Fib @ (cons @ (i 9) @ (cons @ (i 4) @ null))) V.


(cns (i 34) (cns (i 3) null)).


?- eval (equal @ (abs x\x) @ (abs y\y)) V.


type eq    tm -> tm -> o.
eq (i N) (i N)  &  eq tt tt  &  eq ff ff.
eq null null.
eq (cns X Y) (cns U V) :- eq X U, eq Y V.
```

2.2 A specification using evaluation contexts

```
type non_val, redex  tm -> o.
type reduce, evalc   tm -> tm -> o.
type context         tm -> (tm -> tm) -> tm -> o.

% Declare which expressions are not values
non_val (_ @ _) & non_val (fixpt _) & non_val (cond _ _ _).
non_val M :- special _ M.

% Declare which expressions are top-level reducible expressions
redex F       :- special _ F.
redex (U @ V) :- val U, val V.
redex (cond tt _ _) & redex (cond ff _ _) & redex (fixpt _).

% Describe how to reduce a redex
reduce ((abs R) @ N) (R N).
reduce (fixpt R) (R (fixpt R)).
reduce (cond tt L R) L.
reduce (cond ff L R) R.
reduce F (spec C F nil) :- special C F.
reduce ((spec 1 Fun Args) @ N) V :- eval_spec Fun (N::Args) V.
reduce ((spec C Fun Args) @ N) (spec D Fun (N::Args)) :-
```

```
  C > 1, D is C - 1.

% Separate an expression into an evaluation context and a redex
context R (x\ x) R :- redex R.
context (cond M N P) (x\ cond (E x) N P) R :-
  non_val M, context M E R.
context (M @ N) (x\ (E x) @ N) R :-
  non_val M, context M E R.
context (V @ M) (x\ V @ (E x)) R :-
  val V, non_val M, context M E R.

% Evaluation repeatedly uses evaluation contexts and redexes
evalc V V :- val V.
evalc M V :- context M E R, reduce R N, evalc (E N) V.


 ?- context (cond ((abs x\ ff) @ tt) (i 2) (i 3)) E R.


 ?- context (cond ff ((abs x\ i 2) @ (i 3)) (i 4)) E R.
```

3 Manipulating functional programs

3.1 Partial evaluation of miniFP programs

```
?- prog "map" (fixpt Body), Unfold = (Body (fixpt Body)).


abs f\ abs l\
   cond (nullp @ l) null
          (cons @ (f @ (car @ l)) @
                  (fixpt map\ abs f1\ abs l1\
                         cond (nullp @ l1) null
                          (cons @ (f1 @ (car @ l1)) @
                                (map @ f1 @ (cdr @ l1))) @ f @ (cdr @ l)))


type mixeval    tm -> tm -> o.
mixeval (abs R) (abs S) :- pi k\ val k => eval (R k) (S k).


?- prog "appnd" App, eval (App @ (cons @ (i 1) @ (cons @ (i 5) @ null))) R,
   mixeval R S.


abs w\ cons @ (i 1) @ (cons @ (i 5) @ w).
```

3.2 Transformation to continuation passing style

```
type ftrans, phi   tm -> tm -> o.

ftrans (abs V) (abs k\ k @ U) :- phi (abs V) U.
ftrans (M @ N) (abs k\ P @ (abs m\ Q @ (abs n\ m @ k @ n))) :-
    ftrans M P, ftrans N Q.
```

```
phi (abs M) (abs k\ abs x\ (P x) @ k) :-
    pi x\ pi y\ ftrans x (abs k\ k @ y) => ftrans (M x) (P y).


?- ftrans ((abs u\u) @ (abs u\u)) F.

F = (abs W1\ (abs (W2\ W2 @ abs W3\ abs W4\ abs (W5\ W5 @ W4) @ W3)) @
    (abs W2\ (abs (W3\ W3 @ abs W4\ abs W5\ abs (W6\ W6 @ W5) @ W4)) @
    (abs W3\ W2 @ W1 @ W3)))


?- ftrans ((abs x\x) @ (abs x\x)) T, red T S.


(abs W1\ (abs W2\ abs W3\ W2 @ W3) @ W1 @ (abs W2\ abs W3\ W2 @ W3))


type adm                                (tm -> tm) -> tm.
type phi, ftrans, admred, red1, red   tm -> tm -> o.

ftrans (abs V) (adm k\ k @ U) :- phi (abs V) U.
ftrans (M @ N) (adm k\ P @ (adm m\ Q @ (adm n\ m @ k @ n))) :-
    ftrans M P, ftrans N Q.

phi (abs M) (abs k\ abs x\ (P x) @ k) :-
    pi x\ pi y\ ftrans x (adm k\ k @ y) => ftrans (M x) (P y).

admred ((adm R) @ N) (R N).

red1 M N :- admred M N.
red1 (M @ N) (M' @ N) & red1 (N @ M) (N @ M') :- red1 M M'.
red1 (adm R) (abs S)  & red1 (abs R) (abs S)  :- pi x\ red1 (R x) (S x).

red M N :- red1 M P, !, red P N.
red M M.
```

4 Bibliographic notes

```
type     let      tm -> (tm -> tm) -> tm.


eval (let T R) V :- eval T U, eval (R U) V.
```

Chapter 11 Encoding a Process Calculus Language

1 Representing the expressions of the π-calculus

```
kind name       type.

kind proc         type.
type null         proc.
type plus, par    proc -> proc -> proc.
```

```
type in           name -> (name -> proc) -> proc.
type out, match   name -> name -> proc -> proc.
type taup         proc -> proc.
type nu           (name -> proc) -> proc.
type bang         proc -> proc.

type a, b, c    name.
type example    int -> proc -> o.
example 1 (par (in b y\ null) (out b a null)).
example 2 (plus (in b y\ out b a null) (out b a (in b y\ null))).
example 3 (nu x\ par (in x y\ null) (out x a null)).
example 4 (nu x\ out a x null).
example 5 (in a y\ par  (in y w\ null) (out b b null)).
example 6 (in a y\ plus (in y w\ out b b null) (out b b (in y w\ null))).
example 7 (nu y\ out a y (par  (in y w\ null) (out b b null))).
example 8 (nu y\ out a y (plus (in y w\ out b b null)
                               (out b b (in y w\ null)))).
```

2 Specifying one-step transitions

```
one  (match X X P) A P' :- one  P A P'.
onep (match X X P) A P' :- onep P A P'.
one  (plus P Q)    A P' :- one  P A P'; one  Q A P'.
onep (plus P Q)    A P' :- onep P A P'; onep Q A P'.


one  (nu P) A (nu P')           :- pi y\ one  (P y) A (P' y).
onep (nu P) A (x\ nu y\ P' y x) :- pi y\ onep (P y) A (P' y).


kind action       type.
type tau          action.
type up, dn       name -> name -> action.

type one    proc ->          action ->           proc  -> o.
type onep   proc -> (name -> action) -> (name -> proc) -> o.

one  (taup P)     tau      P.
one  (out X Y P)  (up X Y) P.
onep (in X M)     (dn X)   M.
one  (match X X P) A P' :- one  P A P'.
onep (match X X P) A P' :- onep P A P'.
one  (plus P Q) A P'    :- one  P A P'; one  Q A P'.
onep (plus P Q) A P'    :- onep P A P'; onep Q A P'.
one  (par  P Q) A (par P' Q) &
one  (par  Q P) A (par Q P') :- one P A P'.
onep (par  P Q) A (y\ par (P' y) Q) &
onep (par  Q P) A (y\ par Q (P' y)) :- onep P A P'.
one  (nu P) A (nu P')               :- pi y\ one  (P y) A (P' y).
onep (nu P) A (x\ nu y\ P' y x)     :- pi y\ onep (P y) A (P' y).
onep (nu P) (up X) P'               :-
                      pi y\ one  (P y) (up X y) (P' y).
one  (par P Q) tau (nu y\ par (P' y) (Q' y)) &
```

```
one  (par Q P) tau (nu y\ par (Q' y) (P' y)) :-
                              onep P (up X) P', onep Q (dn X) Q'.
one  (par P Q) tau (par S (T Y)) :- one  P (up X Y) S,
                                    onep Q (dn X)   T.
one  (par P Q) tau (par (S Y) T) :- onep P (dn X) S,
                                    one  Q (up X Y) T.


onep (in X M) (dn X) M.
onep (nu P)   (up X) P' :- pi y\ one  (P y) (up X y) (P' y).
one  (par P Q) tau (nu y\ par (P' y) (Q' y)) &
one  (par Q P) tau (nu y\ par (Q' y) (P' y)) :-
                           onep P (up X) P', onep Q (dn X) Q'.
```

3 Animating π-calculus expressions

```
?- example 1 P, one P A P'.
P = par (in b W\ null) (out b a null)
A = up b a
P' = par (in b W\ null) null;

P = par (in b W\ null) (out b a null)
A = tau
P' = par null null;
no

?- example 1 P, onep P A P'.
P = par (in b W\ null) (out b a null)
A = W\ dn b W
P' = W\ par null (out b a null);
no

?- example 3 P, one P A P'.
P = nu W\ par (in W y\ null) (out W a null)
A = tau
P' = nu W\ par null null ;
no

?-


?- example 1 P, trace P Tr.


empty
tr (up b a) empty
tr (up b a) (tr (dn b T) empty)
tr tau empty
tr (dn b T) empty                    % Correction made here
tr (dn b T) (tr (up b a) empty)      % Correction made here


?- trace (in a Y\ plus (match Y b (out Y Y null))
                       (match Y c (out Y Y null))) Tr.
```

```
empty
tr (dn a T) empty
tr (dn a b) (tr (up b b) empty)
tr (dn a c) (tr (up c c) empty)


kind trace  type.
type empty  trace.
type tr     action -> trace -> trace.
type trp    (name -> action) -> (name -> trace) -> trace.

type trace              proc -> trace -> o.
trace P empty.
trace P (tr Act Tr) :- one P Act Q, trace Q Tr.
trace P (trp (up X) Tr) :- onep P (up X) Q,
                           pi x\ trace (Q x) (Tr x).
trace P (tr (dn X Y) Tr) :- onep P (dn X) Q, trace (Q Y) Tr.
```

4 May versus must judgments

```
?- example 1 P, comptrace P Tr.
Tr = tr (up b a) (tr (dn b T) empty)
P = par (in b (W1\ null)) (out b a null);

Tr = tr tau empty
P = par (in b (W1\ null)) (out b a null);

Tr = tr (dn b T1) (tr (up b a) empty)
P = par (in b (W1\ null)) (out b a null);
no

?-


type possible, terminal  proc -> o.
type comptrace           proc -> trace -> o.
type separating_trace    proc -> proc -> trace -> o.
type trace_equiv         proc -> proc -> o.

possible P :- one P _ _ ; onep P _ _.
terminal P :- not (possible P).

comptrace P empty :- terminal P.
comptrace P (tr  Act     Tr) :- one  P Act Q, comptrace Q Tr.
comptrace P (tr (dn X Y) Tr) :- onep P (dn X) P',
                                comptrace (P' Y) Tr.
comptrace P (trp (up X)  Tr) :- onep P (up X) P',
                                pi x\ comptrace (P' x) (Tr x).

separating_trace P Q T :- trace P T,  not (trace Q T).

trace_equiv P Q :- not (separating_trace P Q _),
                   not (separating_trace Q P _).
```

```
?- example 5 P, example 6 Q, separating_trace P Q T.
T = tr (dn a b) (tr tau empty)
P = in a (W1\ par (in W1 (W2\ null)) (out b b null));
Q = in a (W1\ plus (in W1 (W2\ out b b null)) (out b b (in W1 (W2\ null))))
no

?- example 5 P, example 6 Q, separating_trace Q P T.
no

?-


?- example 7 P, example 8 Q, separating_trace P Q T.
no

?- example 7 P, example 8 Q, separating_trace Q P T.
no

?-


type  foreach2   (A -> B -> o) -> (A -> B -> o) -> o.
type  sim        proc -> proc -> o.

foreach2 P Q :- not (sigma X\ sigma Y\ P X Y, not (Q X Y)).

sim P Q :- foreach2 (A\P'\ one P A P')
                    (A\P'\ sigma Q'\ one Q A Q', sim P' Q'),
           foreach2 (A\P'\ onep P A P')
                    (A\P'\ sigma Q'\ onep Q A Q',
                           pi x\ sim (P' x) (Q' x)).


?- sim (in a x\ par (in x y\ null) (out c b null))
       (in a x\ plus (in x y\ out c b null) (out c b (in x y\ null))).
solved

?-
```

5 Mapping the λ-calculus into the π-calculus

```
one  (bang P) A (par P1 (bang P)) :- one P A P1.
onep (bang P) X (y\ par (M y) (bang P)) :- onep P X M.
one (bang P) tau (par (par R (M Y)) (bang P)) :-
  onep P (dn X) M, one P (up X Y) R.
one (bang P) tau (par (nu y\ par (M y) (N y)) (bang P)) :-
  onep P (up X) M, onep P (dn X) N.


type trans  tm -> (name -> proc) -> o.

trans (abs M) (u\ in u x\ in u v\ P x v) :-
   pi x\ pi y\ trans x (u\ out y u null) => trans (M x) (P y).
```

```
trans (app M N)
      (u\ nu v\ par (P v) (nu x\ out v x (out v u (bang (in x Q))))) :-
   trans M P, trans N Q.


?- trans (abs w\ w) P, comptrace (P b) T.
P = u\ in u x\ in u y\ out x y null
T = tr (dn b T1) (tr (dn b T2) (tr (up T1 T2) empty));
no

?- trans (app (abs w\ w) (abs w\ w)) P, comptrace (P b) T.
P = u\ nu u\ par
             (nu y\ out u y (out u u
                 (bang (in y w\ in w r\ in w s\ out r s null))))
             (in u z\ in u v\ out z v null)
T = tr tau (tr tau (tr tau
       (tr (dn b T1) (tr (dn b T2) (tr (up T1 T2) empty)))));
no

?-
```

6 Bibliographic notes

Appendix A The Teyjus System

1 An overview of the Teyjus system

2 Interacting with the Teyjus system

```
% tjsim
Welcome to Teyjus
Copyright (C) 2008 A. Gacek, S. Holte, G. Nadathur, X. Qi, Z. Snow
Teyjus comes with ABSOLUTELY NO WARRANTY
This is free software, and you are welcome to redistribute it
under certain conditions. Please view the accompanying file
COPYING for more information
[toplevel] ?-


[toplevel] ?- pi x:int \ (F x) = (x :: 1 :: x :: nil).

The answer substitution:
F = W1\ W1 :: 1 :: W1 :: nil

More solutions (y/n)? y
no (more) solutions

[toplevel] ?-


[toplevel] ?- pi x:int \ (F x) = (x :: 1 :: x :: nil.
(1,19) : Error : Unmatched parenthesis starting here
[toplevel] ?-
```

```
[toplevel] ?- pi x:int \ (F x) = (x x).
(1,20) : Error : operator is not a function
        operator type: int
        in expression: x x.
[toplevel] ?-


[toplevel] ?- pi x \ (F x) = (x :: x :: nil).
The answer substitution:
F = W1\ W1 :: W1 :: nil
More solutions (y/n)? n
yes
[toplevel] ?-


pi x \ (F x) = (x :: 1 :: x :: nil)


[toplevel] ?- pi x \ (F x) = (g x x).
(1,16) : Error : undeclared constant 'g'
[toplevel] ?-


module lists.

type  append    list A -> list A -> list A -> o.
append nil L L.
append (X::L) K (X::M) :- append L K M.

type rev_aux    list A -> list A -> list A -> o.
rev_aux nil L L.
rev_aux (X::L1) L2 L3 :- rev_aux L1 (X::L2) L3.

type reverse    list A -> list A -> o.
reverse L1 L2 :- rev_aux L1 nil L2.

type member     A -> list A -> o.
member X (X :: L).
member X (Y::L) :- member X L.

end


sig lists.

type append     list A -> list A -> list A -> o.
type reverse    list A -> list A -> o.
type member     A -> list A -> o.

end


% tjcc lists


% tjlink lists
```

```
% tjsim lists
Welcome to Teyjus
Copyright (C) 2008 A. Gacek, S. Holte, G. Nadathur, X. Qi, Z. Snow
Teyjus comes with ABSOLUTELY NO WARRANTY
This is free software, and you are welcome to redistribute it
under certain conditions. Please view the accompanying file
COPYING for more information
[lists] ?- append (1::2::nil) (3::4::nil) L.

The answer substitution:
L = 1 :: 2 :: 3 :: 4 :: nil
More solutions (y/n)? y

no (more) solutions

[lists] ?-


[lists] ?- rev_aux (1::2::nil) nil L.
(1,0) : Error : undeclared constant 'rev_aux'

[lists] ?-
```

3 Using modules within the Teyjus system

```
sig assoclist.
kind pair      type -> type -> type.
type pr        A -> B -> pair A B.
type assoc     A -> B -> list (pair A B) -> o.
type addassoc  A -> B -> list (pair A B) -> list (pair A B) -> o.
end

module assoclist.

accumulate lists.

kind pair      type -> type -> type.
type pr        A -> B -> pair A B.
type assoc     A -> B -> list (pair A B) -> o.

assoc X Y L :- member (pr X Y) L.

type addassoc  A -> B -> list (pair A B) -> list (pair A B) -> o.

addassoc X Y L ((pr X Y)::L).

end


type  member   A -> list A -> o.


module testassoc.
accumulate  lists,assoclist.
```

```
end


sig lists.
exportdef append     list A -> list A -> list A -> o.
exportdef reverse    list A -> list A -> o.
exportdef member     A -> list A -> o.
end


useonly  member   A -> list A -> o.
```

4 Special features of the Teyjus system

4.1 Built-in types and predicates

```
[toplevel] ?- X = "every" ^ "thing".

The answer substitution:
X = "every" ^ "thing"

More solutions (y/n)? y
no (more) solutions

[toplevel] ?- X is "every" ^ "thing".

The answer substitution:
X = "everything"

More solutions (y/n)? y
no (more) solutions

[toplevel] ?-
```

4.2 Deviations from the language assumed in this book

```
?- p a => p b => p X.


test X :- p a => p b => p X.


?- test X.
```

This document was translated from L^AT_EX by HEVEA.